

ReplayCapsule-RV: Event-Sufficient Hardware Failure Capsules for Replaying Embedded RISC-V Interrupt/MMIO Bugs

Anonymous Authors
Anonymous Institution

Abstract—Embedded firmware failures often depend on interrupt timing and memory-mapped I/O values that are visible to software but absent from ordinary crash reports. ReplayCapsule-RV records commit-indexed event-sufficient capsules for a scoped class of single-hart RV32I interrupt/MMIO bugs, then replays those events in a host-driven RTL harness. The implementation integrates a ReplayCapsule recorder with PicoRV32, compiler-built RV32I firmware images, Verilator record/replay execution, corruption checking, runtime summaries, and mapped FPGA synthesis. The generated artifact reports 45/45 compiler-backed full RTL replay PASS rows, 10 replay-critical corruptions rejected with 0 unexpected accepts and 2 not-applicable corruption rows, measured cycle/commit and simulator wall-time overhead summaries, and same-target full-core ECP5 mapped overhead for baseline and ReplayCapsule board wrappers. The current prototype prioritizes replay fidelity and auditability over area minimization: it provides a record-side hardware capsule path and host-driven replay harness, but does not yet implement a synthesizable replay-consume datapath.

I. INTRODUCTION

Embedded firmware often fails because the external world arrives at the wrong architectural moment. A sensor value crosses a threshold, an interrupt lands inside a short critical section, or a command register is sampled before firmware has restored an invariant. These failures are difficult to reproduce after the fact because the decisive facts are boundary events: interrupt delivery and return, MMIO values, and property context. Full instruction or processor-trace mechanisms can preserve far richer execution histories, but many firmware validation failures need a narrower replay object: the events that drive the same firmware-visible path to the same violation.

ReplayCapsule-RV studies that point in the design space. For a deterministic single-hart RV32I platform with the same firmware image and initial architectural state, the recorder captures commit-indexed interrupt/MMIO events, selected boundary observations, and property-failure metadata. The replay harness consumes those events at the same commit boundary and checks property metadata, event payloads, and final signatures. The design is deliberately scoped: it records replay-driving facts for a defined class of embedded failures rather than attempting to replace full trace, snapshots, or platform-wide debug infrastructure.

This paper makes three contributions:

- an event-sufficient capsule model for single-hart RV32I interrupt/MMIO failures;

- a ReplayCapsule-RV hardware recorder and host-driven RTL replay harness;
- an artifact-backed evaluation across compiler-backed firmware, corruption tests, runtime overhead, and mapped full-core FPGA evidence.

The evaluation is intentionally audit-oriented. The artifact reports compiler-backed firmware builds, 45/45 full RTL replay PASS rows, full RTL corrupted-capsule checks, runtime overhead summaries across baseline and recorder configurations, and full-core ECP5 mapped results. Claims outside this scope are treated as limitations rather than inferred results.

II. MOTIVATION

The motivating failures are firmware bugs whose manifestation depends on external ordering rather than arithmetic alone. The benchmark suite covers sensor threshold handling, interrupt races, MMIO ordering, stack protection, UART command handling, and watchdog timeout behavior. Each family has compiler-built failing and fixed firmware images; selected families also include a no-failure edge case. Table I summarizes the generated benchmark set.

The threat model is a debugging and validation setting, not an attacker model. The recorder may observe firmware-visible events and property-checker outputs. Replay starts from the same image and initial architectural state. Unsupported non-determinism is out of scope unless it is modeled as an event. In particular, multicore interleavings, DMA, caches/coherence effects, analog device state, and undefined C behavior are not covered by the current artifact.

III. EVENT-SUFFICIENT REPLAY MODEL

The replay model is defined over architectural transitions. The original run and replay run share the same platform profile, firmware image, and initial architectural state. Between boundary events, the RV32I core semantics are deterministic. At a boundary, replay consumes the recorded event at the same commit index and validates replay-visible observations.

An event contains a commit index, a same-index order tag, a class, and a payload. Core classes are those required to drive replay in the scoped failures: MMIO read values, MMIO write observations, interrupt entry and exit, external inputs, selected stores or branch outcomes when they influence the property, checkpoint/hash values when used, and the property-failure signature. Diagnostic PC context is useful for debugging and

Generated source: paper/figures/replay_flow.svg

Fig. 1. Record/replay timeline with commit-indexed events. Source generated from scripts/make_figures.py.

Generated source: paper/figures/architecture_overview.svg

Fig. 2. ReplayCapsule hardware architecture. Source generated from scripts/make_figures.py.

comparator alignment, but the model does not assert a globally smallest trace.

Figure 1 shows the intended record/replay timeline. The recorder captures only the boundary facts needed to make the replay run take the same firmware-visible path through the target failure prefix. Overflow invalidates replay availability; it is a status bit, not a recoverable compression mode.

The proof sketch in formal/replay_sufficiency_theorem.md is by induction over commit index: equal initial state, deterministic ordinary steps, equal boundary-event payloads at nondeterministic steps, and deterministic property checking over the equal prefix.

IV. HARDWARE ARCHITECTURE

ReplayCapsule-RV instruments the processor boundary rather than replacing the core. The PicoRV32 wrapper exposes memory, MMIO, interrupt, and commit-context signals to the record-side modules. The event tap observes candidate facts, the classifier selects replay-relevant and diagnostic classes, the slicer retains context around a property failure, the capsule buffer stores encoded packets, and the property checker freezes the capsule on a target violation.

The board-level mapped wrappers expose only clock, reset, UART, GPIO, status LEDs, and three recorder-status pins. Large debug and internal buses remain inside the FPGA design. The prototype prioritizes replay fidelity and auditability over area minimization; Section VI reports the generated same-target ECP5 mapped overhead rather than presenting the recorder as an optimized implementation.

V. RECORD/REPLAY IMPLEMENTATION

The replay comparator consumes an expected capsule and an observed replay trace. A passing comparison requires matching property metadata, the required event classes at the selected commit or cycle index, exact payload equality for replay-driving events, and no extra replay-driving events. Negative tests corrupt metadata, remove events, duplicate events, shift indices, swap order tags, and perturb payloads. The same comparator is used for model fixtures and exported RTL-smoke capsules.

The current replay-consume path is host-driven. The Verilator harness records a capsule from the instrumented PicoRV32 wrapper, reruns the same compiler-backed firmware under replay control, injects the recorded interrupt/MMIO boundary facts, and checks the resulting trace. This is strong evidence for the event stream and RTL integration, but it is not a synthesizable replay-consume datapath.

TABLE I

COMPILER-BACKED FIRMWARE BENCHMARK SUITE. SOURCE: RESULTS/PROCESSED/FIRMWARE_BUILD.CSV.

Family	Variants	Failure mechanism
Sensor threshold	failing, fixed, no_failure_edge	MMIO sensor threshold and deadl
Interrupt race	failing, fixed	interrupt delivery inside a critical
MMIO ordering	failing, fixed	unsafe command/output MMIO or
Stack corruption	failing, fixed	protected-store observation reaches
UART command	failing, fixed, no_failure_edge	unsafe UART command drives act
Watchdog timeout	failing, fixed, no_failure_edge	missed heartbeat under long-runni

TABLE II

HOST-DRIVEN FULL RTL REPLAY SUCCESS. SOURCE: RESULTS/PROCESSED/FULL_RTL_REPLAY.CSV.

Evidence	Pass	Total	Firmware source
Record/replay/signature match	45	45	compiler C
Compiler-backed rows	45	45	compiler C

Bounded local checks cover recorder and control contracts such as event classification, buffer behavior, property signatures, replay mismatch handling, register access, hash/signature update, and MMIO/interrupt logging. These checks support local RTL obligations; they are not a mechanized end-to-end processor theorem.

VI. EVALUATION

All tables in this section are generated from processed CSV files by scripts/render_paper_tables.py. The paper-number audit checks that numeric claims in the paper appear in generated CSVs or generated table sources.

A. Replay and Corruption Results

Table II reports 45/45 compiler-backed host-driven full RTL replay rows. Table III reports 10 corrupted-capsule rows rejected, 0 unexpected accepts, and 2 not-applicable rows for corruption classes outside the current commit-index harness.

B. Runtime and Mapping

Table IV reports measured cycle, commit, and simulator wall-time summaries for baseline_no_recorder, recorder_present_disabled, and recorder_enabled. Cycle and commit medians are separate from simulator wall-clock measurements.

Table V reports same-target ECP5 place-and-route for the board-scoped baseline and ReplayCapsule designs. The full-core mapped rows use the same ecp5-85k target, the same Yosys synth_ecp5 plus nextpnr-ecp5 flow, the same memory size, and the same board-level I/O assumptions. The recorder presence check passes on the mapped ReplayCapsule netlist.

VII. RELATED WORK

RISC-V E-Trace, N-Trace, and external debug specifications target processor observability, control-flow reconstruction, and standardized debug transport [1], [2]. ReplayCapsule-RV is complementary: it records commit-indexed replay-driving interrupt/MMIO events for a scoped single-hart RV32I failure

TABLE III
FULL RTL CORRUPTED-CAPSULE BEHAVIOR. SOURCE:
RESULTS/PROCESSED/FULL_RTL_REPLAY_NEGATIVE.CSV.

Result	Rows
REJECT	10
ACCEPT	0
NA	2

TABLE IV
RUNTIME AND SIMULATOR-OVERHEAD SUMMARY. SOURCE:
RESULTS/PROCESSED/RUNTIME_OVERHEAD_SUMMARY.CSV.
SIMULATOR WALL TIME IS NOT A HARDWARE TIMING CLAIM.

Config	Metric	Value
baseline no recorder	sim wall time sec	0.000000
recorder present disabled	sim wall time sec	0.000000
recorder enabled	sim wall time sec	0.000000
recorder present disabled vs baseline no recorder	cycle overhead pct	0.000000
recorder enabled vs baseline no recorder	cycle overhead pct	0.000000
recorder enabled vs baseline no recorder	sim wall time overhead pct	33.950000

TABLE V
FULL-CORE MAPPED ECP5 OVERHEAD. SOURCES:
RESULTS/PROCESSED/MAPPED_SYNTHESIS.CSV AND
RESULTS/PROCESSED/MAPPED_OVERHEAD.CSV.

Design/metric	LUT	FF	BRAM	Fmax MHz
Baseline board	2814	883	6	63.47
ReplayCapsule board	6859	3901	6	50.70
Overhead	143.75%	341.79%	0.00%	-20.12%

TABLE VI
SCOPE AND LIMITATIONS.

Topic	Current scope
Processor	single-hart RV32I/PicoRV32
Non-determinism	45 commit-indexed interrupt/MMIO boundary events
Replay harness	45 host-driven Verilator harness
Replay hardware	45 record-side recorder only; no replay-consume datapath
Memory	45 no multicore, DMA, cache-coherence, or analog-device model
Optimization	45 fidelity and auditability prioritized over area minimization

class. It does not attempt to provide architectural trace coverage or debug interoperability.

Post-silicon trace compression reduces trace bandwidth and storage by exploiting structure in instruction or control-flow streams. ReplayCapsule-RV makes a different tradeoff. It records the boundary events needed by the replay harness, not a compressed representation of the full executed instruction stream.

Deterministic replay systems record nondeterministic inputs and ordering so an execution can be reconstructed [3], [4]. ReplayCapsule-RV borrows that replay discipline but narrows the target to embedded interrupt/MMIO failures, uses commit-indexed hardware capsules, and evaluates a PicoRV32 RTL harness rather than an operating-system or virtual-machine replay substrate.

Runtime verification and assertion monitors detect violations close to the hardware/software boundary [5]. ReplayCapsule-RV uses property checking as the capsule trigger, but its contribution is the replay-driving event stream and the evidence that corrupted capsules fail comparison.

Embedded event logs are widely used to explain firmware failures after deployment. ReplayCapsule-RV differs by making event ordering architectural through commit indices and by validating the capsule through replay comparison instead of presenting the log only as diagnostic text.

FPGA/emulation debugging and checkpointing can capture broad hardware state and rerun workloads under controlled conditions. Snapshot and time-travel systems similarly preserve enough state to resume or inspect execution [6]. ReplayCapsule-RV is narrower: it keeps the same initial state and firmware image, then records the boundary events needed for a selected failure prefix.

The implementation builds on PicoRV32, Verilator, Icarus Verilog, Yosys, nextpnr, and bounded formal tooling [7], [8], [9], [10], [11], [12]. These tools support reproducibility; the

claimed research object is the scoped capsule model, recorder, replay harness, and evidence package.

VIII. LIMITATIONS

The scope is single-hart RV32I firmware with deterministic core semantics, the same program image, the same initial architectural state, and modeled external events. The current theorem excludes multicore interleavings, DMA unless modeled, cache/coherence effects unless modeled, analog non-determinism, and claims about undefined C behavior.

MMIO side effects are modeled only at the replay-visible boundary. Interrupts are injected at an architectural boundary. Replay consumes every required event exactly once; missing, duplicate, reordered, or corrupted events must fail comparison. Capsule overflow invalidates replay availability.

The strongest generated evidence today is compiler-backed firmware, host-driven full RTL replay, full RTL corrupted-capsule rejection, runtime summaries, bounded local formal checks, generic synthesis context, and same-target full-core ECP5 mapped evidence. The artifact does not claim ASIC area or power, multi-hart behavior, DMA behavior, cache-coherence behavior, or a hardware replay-consume datapath.

IX. CONCLUSION

ReplayCapsule-RV shows that a scoped class of embedded RV32I interrupt/MMIO failures can be recorded as commit-indexed capsules and replayed in a host-driven RTL harness with compiler-backed firmware. The artifact reports 45/45 full RTL replay PASS rows, corruption rejection, runtime summaries, and same-target full-core ECP5 mapped overhead. The result is not a replacement for trace standards or snapshots; it is a record-side hardware capsule design and evidence package for replay-driving boundary events. The next step is a synthesizable replay-consume datapath and broader platform modeling, not stronger claims from the current prototype.

Generated source: `paper/figures/replay_success_summary.svg`

Fig. 3. Replay success summary generated from the full RTL replay and corruption ledgers.

Generated source: `paper/figures/runtime_overhead_summary.svg`

Fig. 4. Runtime summary generated from `runtime_overhead_summary.csv`.

REFERENCES

- [1] RISC-V International, “RISC-V Processor Trace Specification,” <https://github.com/riscv-non-isa/riscv-trace-spec>, accessed 2026-06-26.
- [2] —, “RISC-V External Debug Support,” <https://github.com/riscv/riscv-debug-spec>, accessed 2026-06-26.
- [3] T. J. LeBlanc and J. M. Mellor-Crummey, “Debugging parallel programs with instant replay,” in *IEEE Transactions on Computers*, 1987.
- [4] G. W. Dunlap, S. T. King, S. Cinar, M. A. Basrai, and P. M. Chen, “Revirt: Enabling intrusion analysis through virtual-machine logging and replay,” in *OSDI*, 2002.
- [5] M. Leucker and C. Schallhart, Eds., *Runtime Verification*. Springer, 2009.
- [6] S. T. King, G. W. Dunlap, and P. M. Chen, “Debugging operating systems with time-traveling virtual machines,” in *USENIX Annual Technical Conference*, 2005.
- [7] C. Wolf, “PicoRV32: A Size-Optimized RISC-V CPU,” <https://github.com/YosysHQ/picorv32>.
- [8] Verilator Project, “Verilator,” <https://verilator.org/>.
- [9] Icarus Verilog Project, “Icarus Verilog,” <https://steveicarus.github.io/iverilog/>.
- [10] YosysHQ, “Yosys Open SYnthesis Suite,” <https://yosyshq.net/yosys/>.
- [11] —, “nextpnr: portable FPGA place and route,” <https://github.com/YosysHQ/nextpnr>.
- [12] —, “SymbiYosys and Yosys SMTBMC,” <https://yosyshq.readthedocs.io/projects/sby/>.

Generated source: `paper/figures/mapped_overhead_summary`

Fig. 5. Full-core mapped overhead summary generated from `mapped_overhead.csv`.